

XQDot4Z: productos punto cuantizados en un procesador RISC-V

Steve Andy Ildefonso Santos

Resumen—Guardar pesos de red neuronal en cuatro bits ahorra memoria, pero el procesador todavía debe desarmarlos, y la cuantización afín añade un costo fácil de pasar por alto: un código de peso no es un peso, sino un desplazamiento respecto de un punto cero que hay que restar primero. Este trabajo pregunta si esa resta merece construirse dentro de la instrucción de producto punto. Se añade una instrucción fusionada de cuatro términos a Kuntur, procesador RISC-V docente de cinco etapas, y se compara contra tres referencias bajo una política de generación de código congelada: multiplicación por software sin multiplicador, multiplicación escalar, y un producto punto empacado cuya corrección se factoriza, calculando la suma de activaciones una vez por grupo y reutilizándola entre filas. Una campaña registrada de 2280 casos simulados, todos exitosos en dos simuladores independientes, produce un mapa de condiciones y no una sola cifra de aceleración. La fusión reduce ciclos frente a ambas, pero el margen se estrecha al crecer el número de filas, porque la factorizada reparte su corrección entre más, y desaparece con punto cero cero, donde esa alternativa la omite. Cuando varía por fila, la ventaja se paga en tamaño de código: viaja como inmediato, así que cada valor distinto necesita su propio cuerpo de fila, y la variante fusionada ocupa 516 palabras de instrucción frente a 72. Parte del hueco bruto es control de bucle y no aritmética, separados midiendo un gemelo estructural. Son ciclos simulados: no tiempo físico, energía ni área.

Palabras clave—inferencia cuantizada, aritmética de baja precisión, extensiones de instrucciones, verificación de hardware

I. INTRODUCCIÓN

Una capa de red neuronal es, en el fondo, una enorme cantidad de productos punto entre pesos y activaciones. La cuantización permite guardar ambos lados como enteros pequeños con un factor de escala [1], y el ahorro en almacenamiento es notable: ocho pesos de cuatro bits caben en una palabra de 32 bits. Pero ese es un ahorro de memoria, no de trabajo. El procesador todavía tiene que desarmar la palabra antes de poder multiplicar algo, y desempacar no sale gratis.

La cuantización afín añade un segundo costo, más fácil de pasar por alto. Un código de peso de cuatro bits no es el peso. Es un desplazamiento respecto de un punto cero compartido por un grupo de pesos, así que lo que la aritmética necesita de verdad es el código menos ese punto cero. Una resta no es nada. Cuatro restas en cada grupo de cada producto punto de cada fila de una capa sí son algo.

Este trabajo hace una pregunta deliberadamente estrecha sobre esa resta: ¿merece construirse dentro de la instrucción que calcula el producto punto? Esa instrucción se añade a Kuntur, un procesador RISC-V docente de cinco etapas, y se mide contra tres puntos de referencia que resuelven

la misma aritmética de maneras distintas. El objetivo no es un acelerador, ni una pila de inferencia completa. Es un mapa de las condiciones bajo las cuales fusionar la corrección compensa, incluidas aquellas en las que claramente no compensa.

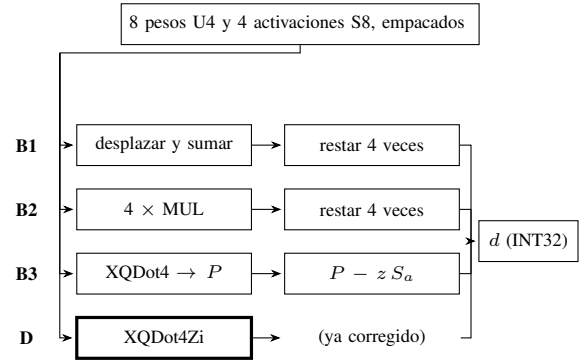


Fig. 1. Trabajo que hace cada variante sobre un mismo grupo de cuatro pesos. B1–B3 son puntos de referencia; D es el diseño en estudio. Las cuatro leen los mismos datos empacados, así que la diferencia no está en el producto sino en quién paga la corrección del punto cero: B1 y B2 la restan término a término, B3 la factoriza y reutiliza S_a entre las N filas, y D la absorbe dentro de la propia instrucción.

II. ANTECEDENTES Y TRABAJO RELACIONADO

Bajo cuantización afín un código entero q representa $r = s(q - z)$, donde $s > 0$ es una escala y z un punto cero. Tomando activaciones cuyo punto cero es cero, el núcleo aritmético en estudio es

$$d = \sum_{i=0}^3 (q_{w,i} - z_w) q_{a,i}. \quad (1)$$

Los pesos se guardan como U4, enteros sin signo de cuatro bits en $[0, 15]$, y las activaciones como S8, enteros con signo de ocho bits en $[-128, 127]$. El resultado es un subtotal INT32 en $[-7680, 7680]$; escalas, sesgo y la acumulación de otros subtotales quedan fuera de la operación.

Leída al pie de la letra, la ecuación (1) cuesta cuatro restas antes de formar producto alguno. Sin embargo puede reordenarse como

$$d = \sum_i q_{w,i} q_{a,i} - z_w \sum_i q_{a,i}. \quad (2)$$

El primer término es un producto punto corriente sobre los códigos en bruto. El segundo es el punto cero por la suma de las activaciones, y esa suma no depende de los

pesos en absoluto. En un producto matriz–vector las mismas activaciones se encuentran con todas las filas, así que puede calcularse una vez y reutilizarse en las N filas. Esa es justamente la razón por la que una comparación justa no puede limitarse a contar las restas de (1): una implementación competente nunca las paga.

Dónde vive el punto cero no es un detalle de este diseño. Los estudios sobre cuantización tratan los esquemas simétrico y asimétrico como un compromiso permanente: forzar el punto cero a cero elimina la corrección por completo y sale más barato en hardware, mientras que conservarlo aprovecha mejor el rango entero en distribuciones sesgadas [2]. Los pesos suelen cuantizarse de forma simétrica justamente por eso, que es lo que vuelve digno de cotizar —en vez de dar por supuesto— el caso de pesos asimétricos. El formato estudiado aquí, pesos de cuatro bits contra activaciones de ocho, es aquel sobre el que han convergido los sistemas de servicio cuantizado en producción [3].

Las extensiones de instrucciones de baja precisión sobre RISC-V son una línea de trabajo asentada. XpulpNN añade instrucciones SIMD de nibble y crumb a un núcleo de clase microcontrolador, y Dustin lleva precisión de bits flexible a un clúster paralelo [4], [5]; Sparq extiende el ISA vectorial con una instrucción de multiplicación, desplazamiento y acumulación para inferencia sub-byte [6]. Los productos punto de pocos bits no son, por tanto, una idea nueva, ni se reclaman como tal aquí. Lo que ninguno de esos trabajos aísla es el costo de la corrección del punto cero en sí, porque la corrección va embebida en un núcleo mayor donde no puede separarse del empaado, del ancho vectorial ni del bucle que la rodea.

La pregunta examinada aquí es más estrecha y, por eso mismo, contestable: dado que un producto punto empaado ya existe, ¿la corrección del punto cero pertenece dentro de la instrucción o fuera de ella?

III. PREGUNTA DE INVESTIGACIÓN Y ALCANCE

¿Bajo qué condiciones fusionar la corrección del punto cero en un producto punto U4 por S8 reduce el costo de ejecución frente a multiplicación escalar y frente a corrección factorizada, considerando la reutilización de activaciones y el costo de suministrar el punto cero?

El alcance es un solo procesador, tamaño de lote uno, y núcleos pequeños de producto punto y matriz–vector. Costo significa ciclos de un pipeline in-order simulado, reportados junto con conteos de instrucciones y tamaño de código. Síntesis, área, frecuencia y energía pertenecen a una etapa posterior con otra metodología, y nada de lo medido aquí se extrapola a ellas.

Una distinción recorre todo lo que sigue. Un punto cero puede ser constante en toda una capa (ZC), variar por fila pero conocerse al generar el código (ZS), o leerse de memoria en ejecución (ZR). La instrucción propuesta lleva el punto cero como inmediato de cuatro bits, lo que cubre los dos primeros regímenes y no el tercero. ZR queda entonces declarado fuera de alcance en vez de estimado, porque un inmediato no puede sustituir sin costo a un valor leído en ejecución.

IV. MÉTODO

A. Las cuatro variantes

Cuatro implementaciones producen las mismas salidas a partir de los mismos datos empaados. Tres son puntos de referencia, nombrados con una B inicial de *baseline*; la cuarta, D, es el diseño en estudio. La Tabla I indica qué las separa y la Fig. 1 muestra qué hace cada una con un solo grupo de cuatro pesos.

TABLA I
LAS CUATRO VARIANTES. B MARCA UN PUNTO DE REFERENCIA Y D EL DISEÑO EN ESTUDIO. LAS CUATRO LEEN DATOS EMPACADOS IDÉNTICOS Y COMPARTEN UNA SOLA POLÍTICA DE GENERACIÓN DE CÓDIGO, ASÍ QUE LO QUE DIFIERE ENTRE ELLAS ES LA ARITMÉTICA, NO EL CÓDIGO QUE LA RODEA.

	Multiplicador	Producto punto	Corrección de z
B1	ninguno	descomposición de bits	por término, sin MUL
B2	MUL escalar	un MUL por peso	por término
B3	MUL escalar	empaado, 4 carriles	factorizada, por fila
D	no requiere	empaado, 4 carriles	dentro de la instrucción

B1 no tiene multiplicador alguno y construye cada producto por descomposición en bits con máscaras acotadas. Es contexto, no una afirmación sobre software óptimo: responde cuánto cuesta esta operación en un procesador sin hardware de multiplicación, y su implementación se congela antes de medir para que no pueda ajustarse después de ver resultados. B2 es la versión directa con multiplicación escalar.

B3 es el competidor serio. Corre una unidad empaada de cuatro carriles sobre los códigos en bruto y aplica la corrección en la forma factorizada de (2), calculando la suma de activaciones una vez por grupo y reutilizándola entre filas. D usa una unidad de cuatro carriles que resta el punto cero dentro de cada carril, así que entrega el subtotal ya corregido y nunca llega a formar una suma de activaciones.

La comparación que responde la pregunta es D contra B3. B1 y B2 acotan el espacio a su alrededor; no la resuelven.

B. Codificación de la instrucción

Las dos instrucciones opcionales son de tipo R y viven en espacio de opcode personalizado, así que ninguna reutiliza una codificación estándar reservada. Escritas en la notación que usan las instrucciones enteras base, donde $\text{add } rd, rs1, rs2$ es $X[rd] = X[rs1] + X[rs2]$ y $\text{lw } rd$ es $X[rd] = M[X[rs1] + \text{signext}(imm)]$, las dos añadidas quedan

$$X[rd] = \sum_{i=0}^3 (U4(X[rs1], i+4h) - z) \cdot S8(X[rs2], i) \quad (3)$$

para $xqdot4zi \ rd, rs1, rs2, z, h, y$

$$X[rd] = \sum_{i=0}^3 U4(X[rs1], i+4h) \cdot S8(X[rs2], i) \quad (4)$$

para $xqdot4 \ rd, rs1, rs2, h$. Aquí $U4(w, j)$ es el j -ésimo campo de cuatro bits sin signo de una palabra y $S8(a, i)$ el i -ésimo byte con signo, h elige qué mitad de los ocho pesos se usa, y el resultado se extiende con signo a 32 bits. Ninguna

toca memoria, ninguna acumula sobre el valor previo de rd , y $rd = x0$ descarta el resultado como de costumbre.

Toda la contribución es el $-z$ de (3). Todo lo que se mide en este artículo se sigue de dónde vive esa resta y de que z sea un inmediato: la Fig. 2 lo muestra ocupando los bits 31:28 de la palabra de instrucción. Esa ubicación es lo que vuelve gratis la corrección en ejecución y, en la misma medida, lo que obliga a un cuerpo de fila distinto por cada punto cero distinto.

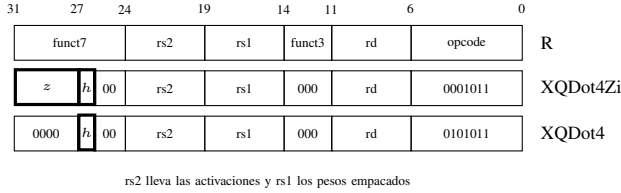


Fig. 2. Formato de las dos instrucciones añadidas, contra el tipo R del ISA base. Ambas son palabras R corrientes en espacio de opcode personalizado: no reutilizan ninguna codificación estándar reservada. Lo que las distingue cabe dentro de funct7. XQDot4Zi gasta cuatro de esos bits en el punto cero z , y ese es exactamente el motivo de que la corrección salga gratis en ejecución y de que el código deba especializarse. XQDot4 deja esos bits en cero y no corrige nada. El bit h elige qué mitad de los ocho pesos se usa.

XQDot4Zi admite 32 combinaciones de z y h ; cualquier otro patrón de funct7 o funct3 en custom-0 se rechaza en vez de ignorarse, y XQDot4 admite solo dos. Rechazar los patrones no implementados importa más de lo que parece: un procesador que los ejecutara en silencio dejaría sin sentido a los controles negativos.

C. Plataforma y verificación

El procesador docente original se conserva y todo el desarrollo ocurre en un clon independiente, de modo que la base no puede desplazarse bajo el experimento. Cada pieza se verifica por separado antes de combinarse.

La unidad XQDot4Zi aislada supera 556 734 comparaciones por simulador contra una referencia en Python con enteros no acotados, junto con ocho controles negativos y cuatro mutaciones inyectadas que las pruebas están obligadas a detectar; el lint no necesita excepciones. La exhaustividad cubre únicamente tuplas de un término. Integrarla al pipeline como XQDot4Zi, en su forma con inmediato, añade 25 programas dirigidos en dos simuladores, comparando retiro, writeback, stores y operandos, y detectando dos mutaciones más.

La MUL escalar opcional que usa B2 es independiente de ese trabajo y devuelve los 32 bits bajos del producto [7]; no implementa ni la extensión M completa ni Zmmul. Supera 86 704 vectores por simulador y 49 programas en cuatro configuraciones. La unidad empacada detrás de B3, que omite la resta del punto cero, supera 67 910 vectores por simulador; la integración de XQDot4 supera 123 programas en ambos simuladores, cubriendo las ocho configuraciones de las tres instrucciones opcionales.

Cada hito queda fijado por SHA-256 en un archivo de estado, cada corrida se repite para mostrar que es determinista,

y se inyectan mutaciones a propósito para demostrar que las pruebas pueden fallar. Dos simuladores independientes, Icarus Verilog y Verilator, deben coincidir exactamente en cada contador antes de registrar un caso como exitoso.

D. Cómo se vuelve honesta la comparación

Una comparación entre una instrucción y su ausencia es fácil de amañar, así que los grados de libertad se eliminan antes de cualquier medición. Desenrollado, asignación de registros, planificación y reutilización de la corrección quedan fijados por una sola política escrita que comparten las cuatro variantes, y esa política, junto con los generadores y las pruebas que la implementan, se congela antes de correr la campaña. La política permite explícitamente omitir trabajo que un punto cero determinado vuelve innecesario, que es como B3 termina eliminando su corrección por completo cuando el punto cero es cero.

Las cuatro variantes se ensamblan sin instrucciones comprimidas [8], de modo que una comparación de tamaño de código cuenta la misma clase de palabra en todos los casos y no premia a la variante que por casualidad comprima mejor.

La rejilla, las semillas, los perfiles de punto cero y las reglas de reporte se registran de antemano, y el registro fija las hipótesis por hash para que no puedan editarse después. Los regímenes nunca se agregan en una sola cifra, y todo caso planificado se conserva, fallos incluidos, porque un caso meramente ausente haría que una campaña incompleta pareciera terminada.

Una sutileza merece mecanismo propio. Como D lleva el punto cero en un inmediato, un punto cero que varía por fila lo obliga a emitir un cuerpo de fila por cada fila, mientras que B3 puede conservar un bucle. Parte de cualquier hueco medido es entonces control de bucle y no corrección. Para separarlos, cada variante se mide también como gemelo estructural: el mismo cuerpo de bucle replicado sin la arista de retorno. Restar el gemelo cotiza el control de bucle con exactitud, y lo que queda es atribuible a la aritmética.

E. Reproducibilidad

Cada hito queda fijado por SHA-256 en un archivo de estado que nombra la evidencia sobre la que descansa, y la puerta de aceptación vuelve a derivar esos hashes en lugar de creerle a la afirmación. Cada verificación corre dos veces y las dos corridas se comparan campo por campo, así que un resultado que dependiera del reloj o del orden no sobreviviría. Se inyectan mutaciones en las unidades, en el banco de pruebas y en el propio mecanismo de registro, y una mutación que ninguna prueba detecta se trata como una prueba que falta. El registro es una cadena en la que cada eslabón fija los bytes del anterior, de modo que un diseño que cambió después de medir no puede presentarse como si hubiera estado fijado de antemano.

Todo el registro es offline y autocontenido: sin red, sin escape a shell, y con todo caso planificado conservado junto a su estado terminal, fallos incluidos. Las fuentes, la evidencia y ambos manuscritos están disponibles en <https://github.com/>

stiffs/xqdot4z, etiquetados en la revisión de la que salieron estas cifras; `make check` vuelve a derivar todos los hashes citados aquí sin re-simular, y `make campaign` reproduce las mediciones desde cero. El código está bajo licencia MIT.

V. RESULTADOS Y DISCUSIÓN

La campaña registrada corrió 2280 casos, todos exitosos en ambos simuladores y sin ninguno ausente. Los ciclos son idénticos entre las diez semillas en las 228 combinaciones de variante, forma y ajuste, y entre las dieciséis fases del perfil de punto cero variable, mientras que las salidas difieren en todas. Eso es una afirmación sobre este núcleo y este layout de datos, no una prueba de independencia frente a datos arbitrarios, y explícitamente no se usa para justificar una rejilla más pequeña.

La Fig. 3 y la Tabla II contienen el resultado central. La ventaja de la fusión sobre B3 cae al crecer el número de filas en todo régimen donde B3 tiene corrección que amortizar, que es la tendencia que el trabajo esperaba. En $z=0$ no hay ventaja alguna: B3 omite tanto la suma de activaciones como la corrección, y las dos variantes coinciden ciclo a ciclo. En $N=1$ un punto cero variable es degenerado, ya que una sola fila no puede tener un punto cero que varíe, así que esas columnas coinciden necesariamente.

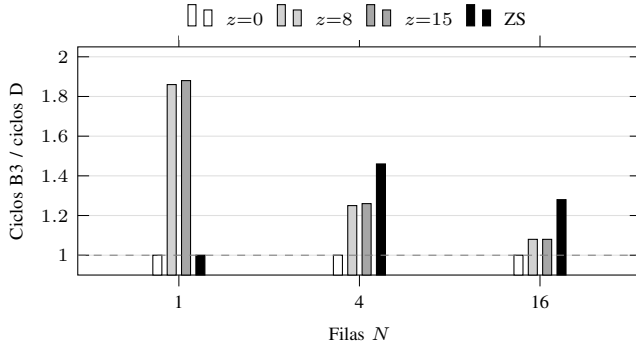


Fig. 3. Ventaja de la fusión por régimen y número de filas. La línea punteada en 1.0 marca la ausencia de ventaja. Los tres primeros grupos son puntos cero constantes; ZS los hace variar por fila. La ventaja se estrecha al crecer N porque B3 amortiza su corrección entre más filas, y en $z=0$ no existe en absoluto porque B3 la omite. En $N=1$ un punto cero variable es degenerado: una sola fila no puede tener un z que varíe. Corrida 20260911T040655320069Z.

TABLA II

CICLOS DE B3 DIVIDIDOS ENTRE CICLOS DE D, POR RÉGIMEN Y NÚMERO DE FILAS. UN VALOR DE 1.00 SIGNIFICA QUE AMBAS VARIANTES CUESTAN EXACTAMENTE LO MISMO. CORRIDA 20260911T040655320069Z, 2280 CASOS; LOS RÉGIMENES NUNCA SE AGREGAN.

N	ZC $z=0$	ZC $z=8$	ZC $z=15$	ZS
1	1.00	1.86	1.88	1.00
4	1.00	1.25	1.26	1.46
16	1.00	1.08	1.08	1.28

La Tabla III descompone el hueco. Todo caso de la campaña satisface la identidad de que los ciclos equivalen a las instrucciones retiradas del núcleo más la profundidad del

pipeline, más los stalls de uso tras carga, más el doble de los flushes por control tomado, y la comprobación la recalcula en vez de leer un total registrado. Aplicada a dieciséis filas con punto cero variable, muestra los 164 ciclos de diferencia entre D y B3 como 110 instrucciones retiradas, 24 ciclos de stall y 30 de flush.

TABLA III

ADÓNDE VAN LOS CICLOS CON $N=16$ Y PUNTO CERO VARIABLE. LOS CICLOS SON LA SUMA DE LAS DEMÁS COLUMNAS MÁS LA PROFUNDIDAD DE CINCO ETAPAS; LA IDENTIDAD SE RECALCULA EN CADA CASO EN VEZ DE REGISTRARSE. D NO PAGA NINGÚN FLUSH PORQUE NO TIENE BUCLE QUE CERRAR.

	Ciclos	Retiradas	Stalls de carga	Flushes
D	581	514	64	0
B3	745	624	88	15
B2	3926	3829	64	15
B1	19813	19716	64	15

La columna de flushes conviene leerla con cuidado, porque no es una virtud. D no paga ningún flush con dieciséis filas únicamente porque no tiene arista de retorno que tomar: su punto cero es un inmediato, así que las filas son código recto. Los quince flushes que paga B3 son el precio de conservar un bucle, y esa misma decisión es la que mantiene pequeño su código. Esta es la diferencia estructural que la medición del gemelo existe para separar de la aritmética.

Con punto cero variable el hueco bruto mezcla dos efectos, y el gemelo estructural los separa. De los 164 ciclos que separan a las variantes en $N=16$, unos 47 pertenecen al control de bucle y 117 a la corrección misma. El cociente corregido pasa de 1.38 a 1.20, no de 1.46 a 1.28. Reportar solo la cifra bruta acreditaría a la instrucción un trabajo que hizo la estructura del bucle.

Frente a MUL escalar, D emplea entre 5.4 y 6.8 veces menos ciclos con $N=16$, con el extremo bajo en $z=0$, donde D no tiene corrección que ahorrar; B1, sin multiplicador, gasta entre 26 y 34 veces más que D.

La especialización no es gratuita, y el costo aterrizó en otra moneda. La Tabla IV da el panorama completo, que resulta menos unilateral de lo que sugiere un solo par de números. Con punto cero constante D es el *menor* de los dos para cualquier número de filas, 39 palabras frente a 68, porque no necesita suma de activaciones ni código de corrección y puede conservar un bucle. Solo cuando el punto cero varía su texto crece con el número de filas, hasta 516 palabras frente a 72, ya que el inmediato exige un cuerpo de fila por cada valor distinto.

TABLA IV

TAMAÑO DE CÓDIGO EN PALABRAS DE INSTRUCCIÓN. CON PUNTO CERO CONSTANTE D ES MENOR QUE B3 Y SE MANTIENE FIJO; CON PUNTO CERO VARIABLE CRECE LINEALMENTE CON EL NÚMERO DE FILAS MIENTRAS B3 NO SE MUEVE. AMBOS EFECTOS SALEN DEL MISMO INMEDIATO.

N	ZC $z=8$		ZS	
	D	B3	D	B3
1	39	68	39	68
4	39	68	132	72
16	39	68	516	72

La ventaja en ciclos y la penalización en tamaño de código son dos caras de una misma decisión de interfaz. Poner el punto cero en un inmediato es lo que vuelve gratis la corrección en ejecución, lo que elimina el bucle y lo que hace crecer el texto. Un diseño que reportara solo la mitad favorable de eso estaría describiendo otra instrucción.

Leídas en conjunto, estas cifras responden la pregunta con una condición y no con un veredicto. La fusión ayuda más donde la corrección no puede amortizarse: pocas filas y punto cero distinto de cero. Ayuda menos, o nada, donde la alternativa puede repartir la corrección entre muchas filas o saltársela por completo. Y donde más ayuda en ciclos, con punto cero variable, es donde más cuesta en código.

A. Amenazas a la validez

La comparación es contra alternativas construidas para este estudio, no contra las extensiones publicadas de la Sección II. Aquellas apuntan a vectores más anchos, a otras precisiones y a núcleos de convolución completos, así que un conteo de ciclos tomado aquí no sería conmensurable con uno tomado allá; la consecuencia es que este trabajo establece dónde ayuda la fusión frente a sus propias alternativas, y no dónde queda frente al estado del arte. El esqueleto compartido reduce el sesgo de optimización pero no demuestra optimalidad ni elimina la interacción entre software y arquitectura. La distribución sintética no representa necesariamente una red real. Con $z=0$ puede omitirse la corrección separada, lo que no demuestra igualdad de rendimiento en general. Despachar entre inmediatos sostendría un punto cero dinámico sin otra codificación, pero aquí no tiene implementación ni costo medido. Los contadores están validados estructuralmente, contra ocho programas dirigidos, tres controles negativos, cuatro mutaciones del banco detectadas y una identidad de contabilidad de ciclos comprobada en cada caso; los ciclos RTL siguen sin demostrar timing físico. La pregunta de sensibilidad queda contestada solo para el número de filas: se midió un grupo de 32 elementos, así que cómo se mueve la comparación con la cantidad de grupos y su tamaño sigue abierto, y el layout de datos congelado no alcanza hoy esas formas.

VI. CONCLUSIONES

Fusionar la corrección del punto cero en la instrucción de producto punto reduce ciclos frente a corrección factorizada y frente a multiplicación escalar, pero no de manera uniforme. El beneficio se estrecha al crecer el número de filas, desaparece cuando el punto cero es cero y, cuando el punto cero varía, se cobra en tamaño de código. La respuesta a la pregunta no es un número sino un mapa de condiciones, y ese mapa es lo que este trabajo aporta.

Quedan fuera el régimen con puntos cero leídos en ejecución, que esta interfaz no puede expresar, y la evidencia física: área, frecuencia y energía requieren síntesis y otra metodología. El hardware adicional que exige la variante fusionada no está contabilizado en ninguna cifra reportada aquí.

REFERENCIAS

- [1] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," arXiv:1712.05877, 2017. [Online]. Available: <https://arxiv.org/abs/1712.05877>
- [2] A. Gholami, S. Kim, Z. Dong, Z. Yao, M. W. Mahoney, and K. Keutzer, "A survey of quantization methods for efficient neural network inference," arXiv:2103.13630, 2021. [Online]. Available: <https://arxiv.org/abs/2103.13630>
- [3] Y. Lin, H. Tang, S. Yang, Z. Zhang, G. Xiao, C. Gan, and S. Han, "QServe: W4A8KV4 quantization and system co-design for efficient LLM serving," arXiv:2405.04532v3. <https://arxiv.org/abs/2405.04532v3>, 2025, primera versión: 2024; revisión consultada: 1 de mayo de 2025.
- [4] A. Garofalo, G. Tagliavini, F. Conti, L. Benini, and D. Rossi, "XpulpNN: Enabling energy efficient and flexible inference of quantized neural network on RISC-V based IoT end nodes," arXiv:2011.14325, 2020. [Online]. Available: <https://arxiv.org/abs/2011.14325>
- [5] G. Ottavi, A. Garofalo, G. Tagliavini, F. Conti, A. D. Mauro, L. Benini, and D. Rossi, "Dustin: A 16-cores parallel ultra-low-power cluster with 2b-to-32b fully flexible bit-precision and vector lockstep execution mode," arXiv:2201.08656v2, 2023. [Online]. Available: <https://arxiv.org/abs/2201.08656v2>
- [6] T. Dupuis, Y. Fournier, M. AskariHemmat, N. E. Zarif, F. Leduc-Primeau, J. P. David, and Y. Savaria, "Sparq: A custom RISC-V vector processor for efficient sub-byte quantized inference," arXiv:2306.09905, 2023. [Online]. Available: <https://arxiv.org/abs/2306.09905>
- [7] RISC-V International, "The RISC-V Instruction Set Manual, Volume I: M extension, version 2.0," ISA v20260120. <https://docs.riscv.org/reference/isa/v20260120/unpriv/m-st-ext.html>, 2026, accessed: 2026-09-08.
- [8] —, "The RISC-V Instruction Set Manual, Volume I: C extension for compressed instructions, version 2.0," <https://docs.riscv.org/reference/isa/unpriv/c-st-ext.html>, 2026, biblioteca de especificaciones; fecha de consulta: 8 de septiembre de 2026. El año identifica la consulta, no la introducción de C.